

Introduction

Welcome to **YShared**! This is a basic example page demonstrating the various Markdown features supported by the documentation.

Headings

Markdown supports multiple heading levels:

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

Text Formatting

You can use **bold text**, *italic text*, and ***bold italic text***.

You can also use ~~strikethrough text~~.

You can highlight `inline code` using backticks.

You can combine formatting, such as **bold with inline code**.

Links

You can create links to other pages:

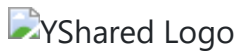
[Getting Started](#)

You can also link to external websites:

[Unity](#)↗

Images

Images can be embedded using:



You can also add alternative text describing the image.

Lists

Unordered List

- Item one
- Item two
- Item three
 - Nested item
 - Another nested item
 - Even deeper

Ordered List

1. First item
2. Second item
3. Third item
 1. Nested item
 2. Another nested item

Task List

- [x] Completed task
 - [] Incomplete task
 - [] Another task
-

Blockquotes

This is a blockquote.

It can span multiple lines.

Blockquotes can also be nested.

Code

Inline code:

```
Debug.Log("Hello World");
```

C# Code Block

```
using UnityEngine;

public class Example : MonoBehaviour
{
    private void Start()
    {
        Debug.Log("Hello World!");
    }
}
```

Although....

Getting Started

Quick Start

Welcome to **YShared**, a feature complete Dev Console system for Unity.

To get started, add the attribute [YCommand](#) to any function.

```
public class MyConsoleCommands : MonoBehaviour
{
    [YCommand]
    public int Add(int a, int b)
    {
        return a + b;
    }
}
```

and that's it! The command "add a:int b:int" is now added to the console.

YCommand Arguments

If you want the name of the console command to be different than the name of the function, you may change the name. You can also add a command description, which will be shown when:

- Using the `help` command
- Disambiguating between commands with the same name but different signature
- Listing the usage of multiple subcommands
- The user is informed there is an error in their input.

```
public class MyConsoleCommands : MonoBehaviour
{
    [YCommand("multiply", "Multiply two numbers together and return the output.")]
    public int SuperComplicatedMathematicalOperation(int a, int b)
    {
        return a * b;
    }
}
```

If the function is a static method, the command will simply run without an instance.

If the function is an instance method, an [ObjectFindType](#) value can be given to tell YConsole how to find the object to run this command on. By default, `ObjectFindType.All` is used.

Value	Description
All	Finds all objects of this class and calls the method on each of them. Internally uses <code>GameObject.FindObjectsByType</code> .
Any	Finds any object of this class and calls the method on it. Internally uses <code>GameObject.FindAnyByType</code> .
First	Finds the first object of this class and calls the method on it. Internally uses <code>GameObject.FindFirstByType</code> .
Argument	Uses the first argument of the command as the instance to run the method on. The first argument does not need to be explicitly typed as the class; if it is, YConsole automatically fills it in.
ArgumentArray	Uses the first argument of the command as an array of instances to run the method on. The first argument does not need to be explicitly typed as an array of the class; if it is, YConsole automatically fills it in.

Built in Commands

YShared comes with several built-in commands to help with getting started.

Command	Description
<code>clear</code>	Clear the console output.
<code>help (page:int)</code>	Prints all commands, their usage format and brief description to the console.
<code>help (cmd:Command)</code>	Shows the manual of a specific command.
<code>screenshot</code>	Take a screenshot and save it to your clipboard.
<code>yconsole fontsize</code>	Change the font size of the console.
<code>yconsole unity_console info</code>	Toggle whether regular Unity Console messages (such as through <code>Debug.Log</code>) are visible in YConsole. Defaults to <code>True</code> .
<code>yconsole unity_console warnings</code>	Toggle whether regular Unity Console messages (such as through <code>Debug.LogWarning</code>) are visible in YConsole. Defaults to <code>False</code> .
<code>yconsole unity_console errors</code>	Toggle whether regular Unity Console messages (such as through <code>Debug.LogError</code>) are visible in YConsole. Defaults to <code>True</code> .

Suggestion Modifiers

Suggestion modifiers are a list of attributes that modify how the autocompletion list (the suggestions) are generated when entering commands through YConsole.

Exclude

By default, most value types automatically autocomplete to a list of suggestions that make sense for that value type. For example, the following enum,

```
public enum Gamemode
{
    Unset,
    Survival,
    Adventure,
    Creative,
    Spectator
}
```

automatically knows that it can autocomplete to those 5 values. So if you have a command such as,

```
public class MyPlayer : MonoBehaviour
{
    [YCommand("gamemode", "Change the current player's gamemode.")]
    public int SuperComplicatedMathematicalOperation(Gamemode gm)
    {
        // Complicated Gamemode change logic here
    }
}
```

YConsole will conveniently suggest the possible, easy to read string values of that enum to you. You could also use the numerical values of the enum to use this command if you want, but YConsole makes it easy by providing you with the list and convenience of autocompletion.

But there is something missing here! Or rather, there is something extra. In our game, `Gamemode.Unset` is the "unassigned" gamemode. Let's say it's used when the player first loads in to our server, but it's not a gamemode we are ever meant to be in to play the actual game. How can we prevent `Unset` from being selectable in this command?

By simply using `[Exclude(nameof(Gamemode.Unset))]` as a parameter attribute.

```
public class MyPlayer : MonoBehaviour
{
```

```

[YCommand("gamemode", "Change the current player's gamemode.")]
public int SuperComplicatedMathematicalOperation([Exclude(nameof(Gamemode.Unset))]
Gamemode gm)
{
    // Complicated Gamemode change logic here... but now without the possibility
of Unset
}
}

```

"Unset" is no longer suggested when trying to use the command "gamemode". (Of course, `[Exclude("Unset")]` would have also worked.)

Exclude accepts a `param string[] exclude_list`, so you can put as many excluded strings in there as you want.

Include

```

public class MyPlayer : MonoBehaviour
{
    [YCommand("set_name", "Change the current player's current name")]
    public string SetName([Include("Player A", "Player B", "Player C")] string new_name)
    {
        // Complicated Gamemode change logic here
    }
}

```

If the function is a static method, the command will simply run without an instance.

If the function is an instance method, an [ObjectFindType](#) value can be given to tell YConsole how to find the object to run this command on. By default, `ObjectFindType.All` is used.

Value	Description
<code>[Include(include_list: string[])]</code>	Adds the list of strings to suggestions.
<code>[Exclude(exclude_list: string[])]</code>	Removes the list of strings from the suggestions.
<code>[Override(override_list: string[])]</code>	Overrides the list of suggestions with the given list.
<code>[NoSuggestions]</code>	Removes all suggestions.
<code>[SuggestionRange(int start_inclusive, int</code>	Generates and adds a suggestions list of all integers from <code>start_inclusive</code> to <code>end_exclusive</code> in increments of <code>step</code> .

Value	Description
<code>end_exclusive, int step = 1)]</code>	
<code>[IncludeDynamic(functionName: string, bool cacheCompatible = false)]</code>	Takes the name of a static function that takes no parameters and returns a string array or IEnumerable. At runtime this function is called to generate a suggestion list and is added to existing suggestions. If <code>cacheCompatible</code> is false, this function is called every time this command is typed, if true, this function is called once and the suggestion list is cached.